# CS336 作业 3：缩放定律

版本 26.0.5 · CS336 课程组 · 2026 年春季

译文说明：完整翻译原作业说明，保留题目标识、分值、实验预算和提交要求。代码与运行输出按原 PDF 保留；本文件不包含习题解答。

# 1 作业概述

本次作业将带你动手研究语言模型的缩放定律（scaling laws）。

## 情景设定

你负责训练 ClosedAI 的下一代语言模型。训练它的开销相当于一个小国的 GDP，还会融化一片南极冰架，因此你希望确保这件事做对了。具体来说，你获得一笔固定计算预算，以实际运行时间（wall-clock time）计量。目标是在该预算下训练出训练损失最低的模型，即计算最优（compute-optimal）的模型。为此，需要在“训练更大的模型”和“用更多 token 训练”之间找到最佳权衡。

缩放定律以经验规律描述语言模型训练损失、模型规模与训练计算量之间的关系，常被用来预测这种权衡 [J. Kaplan 等，2020；J. Hoffmann 等，2022]。本作业要构建缩放定律，估计在 48 B200 小时预算下训练时，计算最优的模型规模及相应超参数。

你不需要自行训练模型，而是向训练 API 提交：(1) 模型超参数，例如 Transformer 层数、嵌入维度、注意力头数、批大小、学习率、训练 token 数等；(2) 本次实验允许的最长实际运行时间。API 会返回使用相应配置训练得到的验证损失。

拟合缩放定律另有 12 B200 小时预算，相当于大型训练运行预算的 25%。API 接受很广的超参数范围，你需要判断应探索搜索空间中的哪些区域，才能有效利用这笔预算。

## 代码与说明

本作业说明位于：https://github.com/stanford-cs336/assignment3-scaling 。请使用 Git 克隆仓库。若有更新，课程组会通知你，届时用 git pull 获取最新版。

## 如何提交

向 Gradescope 提交：

• writeup.pdf：完整说明拟合缩放定律的方法，以及如何用它预测给定 FLOPs 预算下的最优模型规模。说明应详细到足以复现结果。

• code.zip：包含你为拟合缩放定律与计算估计值编写的全部代码。

另外，向 API 提交预测的最优超参数和最终验证损失。作业的一部分成绩取决于你预测的最优模型的实际表现。

## 关于 AI 工具的声明

AI 能自主完成作业中的许多部分，但这会妨碍你深入参与课程内容并从中学习。

允许使用 AI 回答高层次概念问题，或获取函数签名、库 API 等底层编程文档。但是，不允许用 AI 实现任何作业的任何部分。这包括编码智能体，例如 Cursor Agents、Codex、Claude Code，也包括 AI 自动补全，例如 Cursor Tab、GitHub Copilot。使用 AI 智能体时，确保其遵循提供的 AGENTS.md；使用聊天机器人时，也应把相应提示词一并提供。

强烈建议完成作业时关闭 IDE 中的 AI 自动补全。非 AI 自动补全，例如自动补齐函数名，完全可以使用。往届学生反馈，关闭 AI 补全有助于更深入地学习。

完整 AI 政策见：https://docs.google.com/document/d/1SZAlExB1qAc9izHt54gwunNpjKE6wXb8Y7yA_e-baK8/edit?tab=t.0 。

# 2 缩放定律回顾

先回顾 Chinchilla 论文 [J. Hoffmann 等，2022] 中的一种缩放定律拟合方法。核心问题是：给定训练大型语言模型的计算预算 C，选择什么超参数，例如模型规模、训练 token 数，能让训练损失最低？主要难点在于如何从小规模实验外推到大规模。作业第二部分也可以借鉴其他文献的方法，例如 J. Kaplan 等 [1]、G. Yang 等 [3]。

## 2.1 从等 FLOPs 曲线拟合缩放定律

回顾：一个 N 参数的 Transformer，在 D 个 token 上训练，计算预算约为 C=6ND。文献 [2] 的 IsoFLOPs 方法如下：对每个计算预算 C，训练若干规模 N 不同的语言模型，数据量取 D=C/(6N)，得到最终训练损失 L。

这样会产生一组计算量都为 C_i、但模型规模 N_ij 不同的运行。文献 [2] 观察到，固定预算 C_i 时，最终训练损失 L_ij 与模型规模 N_ij 之间呈二次关系。直觉是：N_i 极小时，无论投入多少计算，模型都拟合不了数据，最终损失较高；随着模型增大，损失平滑下降；但超过某个点后，模型过大，预算内无法执行足够多梯度步骤来有效训练。极端情况下，当 N_i 趋于无穷时，连一步梯度更新都超出预算 C，训练只能停在很高的损失。

因此，先确定每个 C_i 下的最优模型和数据集规模，再拟合幂律，根据目标 C 预测 N 与 D；目标预算通常大于此前实验的预算。

具体来说，对预算 C_i 下的运行集合，即它的“IsoFLOPs 曲线”，找出使训练损失最小的模型规模 N_opt(C_i)。由此得到一组数据点〈C_i,N_opt(C_i)〉，以及对应的 D_opt 数据点。用它们拟合 N_opt∝C^a 和 D_opt∝C^b，并外推目标预算下计算最优的模型与数据集规模。

### 题目（chinchilla_isoflops）：IsoFLOPs 缩放定律（5 分）

编写脚本，用一组训练运行的最终训练损失，复现上述 IsoFLOPs 拟合方法。本题使用 data/isoflops_curves.json 中的合成训练数据。该文件是 JSON 数组，每个对象描述一次训练运行，前两条示例如下：

```
[
    {
        "parameters": 49999999,
        "compute_budget": 6e+18,
        "final_loss": 7.192784500319437
    },
    {
        "parameters": 78730505,
        "compute_budget": 6e+18,
        "final_loss": 6.750171320661809
    },
  ...
]
```

原文第 3 页

拟合时可以使用 scipy，尤其是 scipy.optimize.curve_fit，也可以选择任意其他曲线拟合方法。文献 [2] 对每条 IsoFLOPs 曲线拟合二次函数以寻找最小值；本题则建议直接取各预算下训练损失最低的那次运行作为最小值。

(a) 展示外推得到的计算最优模型规模，以及用于拟合的〈C_i,N_opt(C_i)〉点。预算为 $10^{23}$ FLOPs 时，预测最优模型有多大？$10^{24}$ FLOPs 时呢？

提交内容：一张图，展示模型规模随计算预算变化的缩放定律、用于拟合的数据点，并至少外推到 $10^{24}$ FLOPs。再用一句话给出预测的最优模型规模。

(b) 展示外推得到的计算最优数据集规模，以及训练运行对应的〈C_i,D_opt(C_i)〉点。预算分别为 $10^{23}$ 和 $10^{24}$ FLOPs 时，最优数据集规模是多少？

提交内容：一张图，展示数据集规模随计算预算变化的缩放定律、拟合数据点，并至少外推到 $10^{24}$ FLOPs。再用一句话给出预测的最优数据集规模。

# 3 构建缩放定律

在固定预算内，通过第 3.2 节的训练 API 获取运行结果并拟合缩放定律。主要目标是选择模型规模和超参数，使 48 B200 小时训练后的验证损失最低；同时预测该模型将达到的验证损失。

建议分析超参数在较小规模实验中如何影响验证损失，据此为预测的最优模型设置超参数。开始前务必仔细规划实验：一旦超出 12 B200 小时的缩放定律实验预算，API 将拒绝后续请求。

## 3.1 训练运行细节

训练 API 会在 B200 GPU 上运行真实训练任务。提交实验后，实验进入共享队列。排队或运行中的实验，会从 12 小时预算中预留其全部 max_runtime_seconds。实验完成或失败后，API 根据实际报告的运行时间重新计算预算，计费时间至少 1 秒，至多 max_runtime_seconds。例如预留 30 分钟而实际 12 分钟完成，未用的 18 分钟会返还。超时运行按 max_runtime_seconds 计费。

每个训练任务分为 n_evals 段，每段训练 total_train_tokens/n_evals 个 token，之后评估验证集，并在 status.val_losses 中报告验证损失序列。完成的实验以序列最后一项作为最终验证损失。超时失败的实验可能返回部分验证损失，但不算已完成运行。

数据顺序固定。训练数据是已分词的 DCLM 数据，最多 500B token，不会重复遍历多个 epoch。各配置均消耗同一个确定性 token 顺序。验证集使用 $2^{18}$ 个 token，最终排行榜评估会使用更多。model_seed 只控制模型初始化，不影响数据顺序。

模型架构大体与作业 1 相同，只有少量差异。仓库 https://github.com/stanford-cs336/assignment3-scaling 提供模型与训练代码供参考；你应只通过 API 训练模型。

模型使用词表大小 32K 的已分词 DCLM 数据，上下文长度 512。API 支持 AdamW 与 SGD，两者都使用预热后余弦衰减的学习率计划。下方示例采用 AdamW，权重衰减 0.01、梯度裁剪 1.0、预热比例 0.05、最终学习率比例 0.1。损失是在长度 512 的序列上，对下一 token 交叉熵取平均。API 模型没有 dropout。

## 3.2 训练 API

用训练 API 查询缩放定律实验的最终验证损失。API key 是你的 8 位 Stanford ID，例如 06123456。必须连接 Stanford 网络，可能需要 VPN。首先检查能否打开 http://hyperturing.stanford.edu:8000/docs ，再用 /budget 端点验证 key 是否有效。

所有 API 请求都应在 X-API-Key 请求头中包含密钥：

```
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> API_KEY = <YOUR_API_KEY_HERE>
>>> headers = {"X-API-Key": API_KEY}
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

原文第 4 页

提交实验进入队列，轮询状态，完成后读取验证损失。API 提供以下端点。

### POST /submit

提交训练配置，将运行加入队列，返回实验 ID。请求体为 JSON 对象，包含：

• architecture_config：描述 Transformer 架构的对象，包含 attention_bias、head_dim、hidden_size、intermediate_size、num_attention_heads、num_hidden_layers、num_key_value_heads、rms_norm_eps、rope_theta、tie_word_embeddings、dtype、vocab_size。dtype 必须为 "float32" 或 "bfloat16"。

• optimizer_config：优化器配置对象，支持 AdamW 和 SGD，下例使用 AdamW。

• train_batch_size：整数训练批大小。

• val_batch_size：整数验证批大小。

• n_evals：训练期间的验证评估次数。

• total_train_tokens：训练 token 总数。

• max_runtime_seconds：为实验预留的最长实际运行时间，范围为 1 秒至 12 小时。

• model_seed：模型初始化随机种子。

API 将 seq_len 固定为 512，将 n_validation_tokens 固定为 $2^{18} \approx 262k$，并检查一致性条件。例如 hidden_size 必须等于 num_attention_heads * head_dim；num_attention_heads 必须能被 num_key_value_heads 整除；total_train_tokens 必须能被 512 * train_batch_size 整除。

响应 JSON 包含 experiment_id（排队实验的整数 ID）和 budget_summary（包含 used_seconds、remaining_seconds、total_budget_seconds）。

重复提交相同训练配置会返回 409，不额外预留预算。max_runtime_seconds 超过剩余预算时返回 400。HTTP POST 调用示例：

```
>>> import requests
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> headers = {"X-API-Key": <YOUR_API_KEY_HERE>}
>>> training_config = {
...     "architecture_config": {
...         "attention_bias": False,
...         "head_dim": 64,
...         "hidden_size": 448,
...         "intermediate_size": 1280,
...         "num_attention_heads": 7,
...         "num_hidden_layers": 9,
...         "num_key_value_heads": 7,
...         "rms_norm_eps": 1e-6,
...         "rope_theta": 1_000_000,
...         "tie_word_embeddings": False,
...         "dtype": "bfloat16",
...         "vocab_size": 32_000,
...     },
```

原文第 5 页

```
...     "optimizer_config": {
...         "lr_scheduler": {
...             "peak_value": 3e-4,
...             "final_lr_frac": 0.1,
...             "warmup_frac": 0.05,
...             "init_value": 0.0,
...         },
...         "weight_decay": 1e-2,
...         "beta1": 0.9,
...         "beta2": 0.95,
...         "eps": 1e-8,
...         "eps_root": 1e-8,
...         "grad_clip_norm": 1.0,
...     },
...     "train_batch_size": 128,
...     "val_batch_size": 32,
...     "n_evals": 16,
...     "total_train_tokens": 1_048_576,
...     "max_runtime_seconds": 30.0,
...     "model_seed": 0,
... }
>>> requests.post(f"{API_BASE_URL}/submit", headers=headers, json=training_config).json()
{'experiment_id': 1, 'budget_summary': {'used_seconds': 30.0, 'remaining_seconds': 43170.0,
'total_budget_seconds': 43200.0}}
```

原文第 6 页

## GET /budget

根据 API key，返回实验已用或预留的实际时间预算，JSON 字段包括：used_seconds 为已用或预留秒数；remaining_seconds 为剩余秒数；total_budget_seconds 为总秒数。HTTP GET 示例：

```
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

原文第 6 页

## GET /experiments

返回该 key 提交过的全部实验。每个实验含 experiment_id（由 /submit 返回）、training_config（提交的配置）、status（当前状态）。status.status_type 可为：

• "queued"：等待运行。

• "running"：正在运行；状态含 val_losses，即迄今观察到的验证损失。排队之后即进入 running；该阶段包含启动和被抢占的时间，因此显示的 running 时长可能超过设置的最大运行时间。

• "completed"：已完成；状态含 used_runtime_seconds、val_losses、completed_at。

• "failed"：失败或超时。超时失败会包含可供检查的 partial_val_losses，但失败的实验已经崩溃或超时。

HTTP GET 示例：

```
>>> requests.get(f"{API_BASE_URL}/experiments", headers=headers).json()
[{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'queued',
'queued_at': '...'}}]
```

原文第 7 页

### GET /experiment/{experiment_id}

返回单个实验，格式与 /experiments 列表中的元素相同。用它轮询，直到 status.status_type 为 "completed"：

```
>>> requests.get(f"{API_BASE_URL}/experiment/1", headers=headers).json()
{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'completed',
'val_losses': [3.1, 2.9, ...], ...}}
```

原文第 7 页

对完成的实验，使用 status.val_losses 的最后一个值作为最终验证损失。

### POST /final_submission

提交预测的最优超参数和最终验证损失。请求体包含 training_config（预测的最优配置）和 predicted_final_loss（该配置预测的最终验证损失）。可以重复提交，最新提交覆盖旧提交。

```
>>> final_submission = {
...     "training_config": training_config,
...     "predicted_final_loss": 2.75,
... }
>>> requests.post(f"{API_BASE_URL}/final_submission", headers=headers,
json=final_submission).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

原文第 7 页

### GET /final_submission

返回当前最终提交；尚未提交时返回 None。

```
>>> requests.get(f"{API_BASE_URL}/final_submission", headers=headers).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

原文第 7 页

## 3.3 排行榜

使用 API 为 48 B200 小时训练选择配置，目标是让最终模型验证损失最低。

### 题目（scaling_laws）：构建缩放定律排行榜（50 分）

构建缩放定律，据此选择预期在 48 B200 小时预算内达到最低验证损失的模型规模和超参数，同时预测最终验证损失。拟合时通过 API 查询不同配置的最终验证损失，见第 3.2 节。用于拟合的实验不得超过 12 B200 小时，这是 API 强制执行的硬上限。

提交内容：一份排版好的报告，完整描述拟合方法、如何用缩放定律预测给定 FLOPs 预算下的最优模型规模，以及预测值。说明重要设计决策的理由，详细程度应足以复现方法和结果。

对于 48 B200 小时运行可报告的超参数，基本不设限制。所有运行均在单张 B200 上进行。

至少应考虑以下问题，并在报告中解释每项决策：

• 在固定 12 B200 小时拟合预算下，如何选择查询哪些训练运行？

• 如何拟合缩放定律？描述具体方法。熟悉文献 [1] 和 [2] 的方法可能很有帮助。

• 缩放定律对实验数据的拟合程度如何？

• 对给定的 48 B200 小时预算，预测的最优模型规模和损失是多少？

• 若训练预测的最优参数量模型，应使用什么超参数？给定模型配置时，用 12·n_layer·d_model² 估计非嵌入参数量。

除报告外，还要向 API 提交：(1) 预测的最优超参数；(2) 模型的验证损失。部分成绩取决于预测最优模型的表现。

译注：原文在部分段落中将 B200 小时称为“FLOPs 预算”；本译文保留其任务含义，实际 API 按运行秒数记录和限制预算。

# 参考文献

[1] J. Kaplan 等. “Scaling Laws for Neural Language Models.”（神经语言模型的缩放定律。）2020。

[2] J. Hoffmann 等. “Training Compute-Optimal Large Language Models.”（训练计算最优的大型语言模型。）2022。

[3] G. Yang 等. “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer.”（Tensor Programs V：通过零样本超参数迁移调优大型神经网络。）2022。